Answer to the Q. No: 1(a)

Given Infix Expression:

$$(A \times B + C) \div (D - E) \times (F + G) - H$$

Step by step conversion:

Symbol	Stack	Postfix
(	(	
A	(	A
×	(×	A
B	(×	AB
÷	(+	AB×
C	(+	AB×C
)		AB×C+
÷	÷	AB×C+
(	÷(	AB×C+
D	÷(	AB×C+D
−	÷(−	AB×C+D
E	÷(−	AB×C+DE
)	÷	AB×C+DE−
×	×	AB×C÷DE−÷
(	×(	AB×C÷DE−÷
F	×(	AB×C+DE−÷F
+	×(÷	AB×C÷DE−÷F
G	×~~(+~~	AB×C÷DE−÷FG
)	×	AB×C+DE−÷FG+×
−	−	AB×C+DE−÷FG+×
H	−	AB×C÷DE÷FG+×H

∴ Final Expression:

AB×C+DE−÷FG+×H−

Ans. to Q. NO: 1(b)

Given post-fix Expression:

AB * CD + ÷ EF + *

Given Value

A = 6 B = 4 C = 8 D = 4 E = 5 F = 3

Step wise Stack Evaluation:

Symbol	Operation	Stack
A	Push 6	6
B	Push 4	6, 4
*	6×4 = 24	24
C	Push 8	24, 8
D	Push 4	24, 8, 4
+	8+4 = 12	24, 12
÷	24 ÷ 12 = 2	2
E	Push 5	2, 5
F	Push 3	2, 5, 3
+	5+3 = 8	2, 8
*	2×8 = 16	16

∴ Final Result = 16

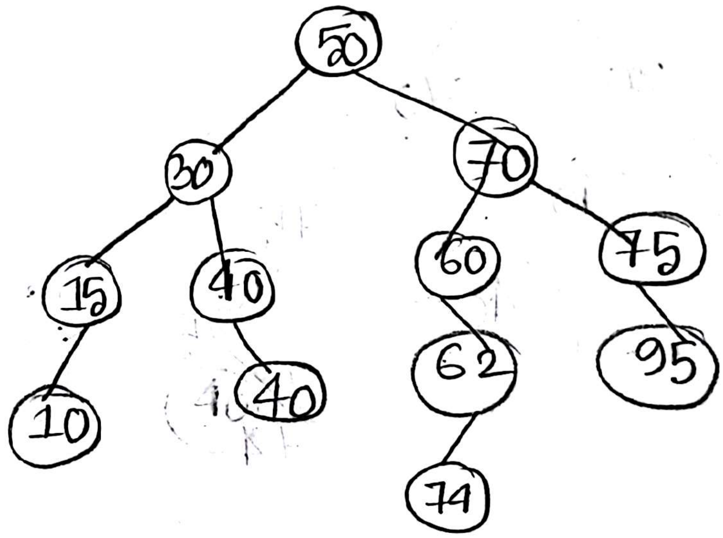
(2)

Answer to the Question No: 2(a)

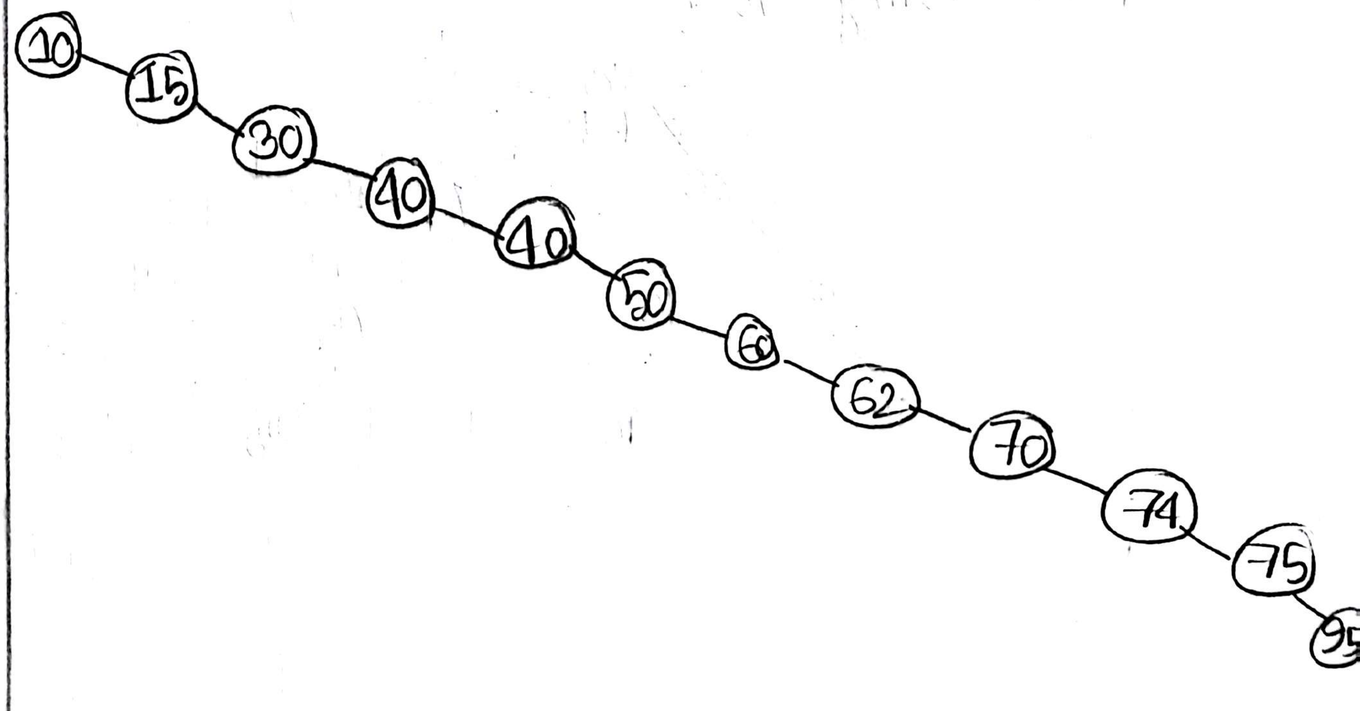
Given Sequence: i) 50, 30, 70, 15, 40, 75, 74, 95, 10, 40, 60, 62

ii) 10, 15, 30, 40, 40, 50, 60, 62, 70, 74, 75, 95

Tree 1 for sequence (i):



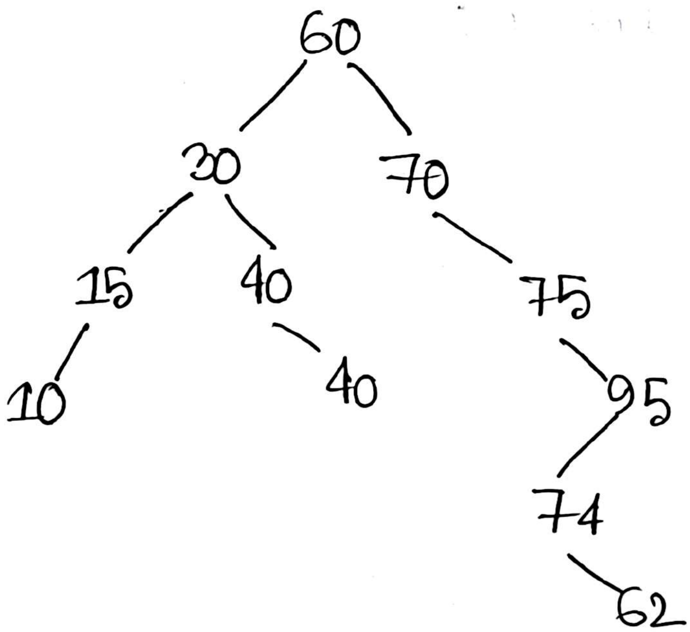
Tree for sequence (ii)



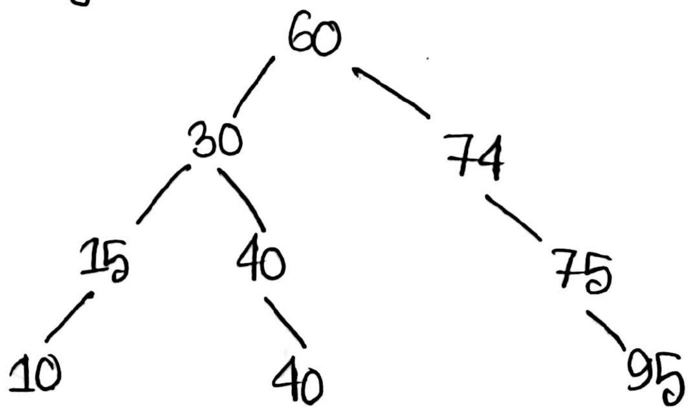
Answer to the Q. NO: 2 (b)

After Deleting 50 · with showing all the tree

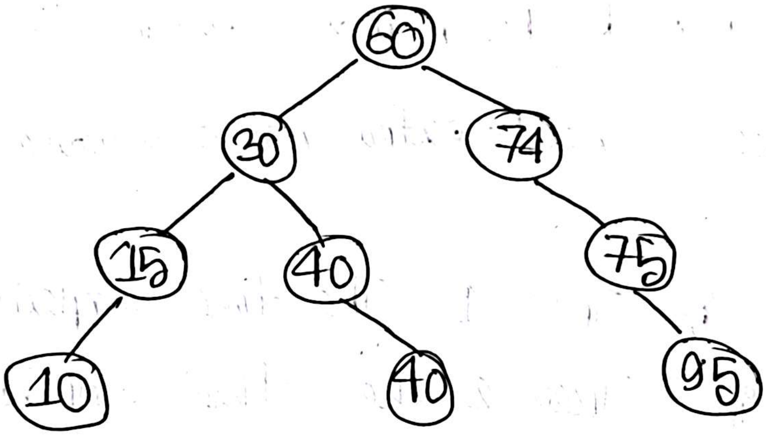
1st step:



Now, Deleting 70:



Final Tree After Deleting these 2 element



Answer to the Question No: 2(c)

we know, In a binary Search Tree The minimum value is found by going to the leftmost node. The maximum value is found by going in the rightmost node.

Tree (1) Before Deletation;

- Minimum value : $50 \rightarrow 30 \rightarrow 15 \rightarrow 10$

  moves = 3.

- maximum value; $50 \rightarrow 70 \rightarrow 75 \rightarrow 95$

  moves = 3

Tree (2) :

Minimum value = Root (1)

∴ moves = 0.

- Maximum value = Path = $10 \rightarrow 15 \rightarrow 30 \rightarrow 40 \rightarrow 40 \rightarrow 50 \rightarrow 60 \rightarrow$
  $62 \rightarrow 70 \rightarrow 71 \rightarrow 75 \rightarrow 95$

∴ moves = 11

so, Tree 1 needs fewer moves because the tree is balanced and Tree-2 needs extra moves because the tree is Skewed.

for that in Tree 1 The time complexity is O(log n) and in Tree 2 the time complexity is O(n)

∴ finding min & max is faster in Tree 1

Answer to the Question NO: 2(d)

In a Binary Search Tree, the height value is always found at the rightmost node. To find the second height's value without deleting any node we frist traverse the tree from the root by repeatedly moving to the right child until the largest node is reached. If this largest node has a left sub-tree, then the second height's value will be the maximum value in that left subtree, which can be found by moving to the rightmost node of the left subtree. If the largest node does not have a left child, then its parent node is second heights value.

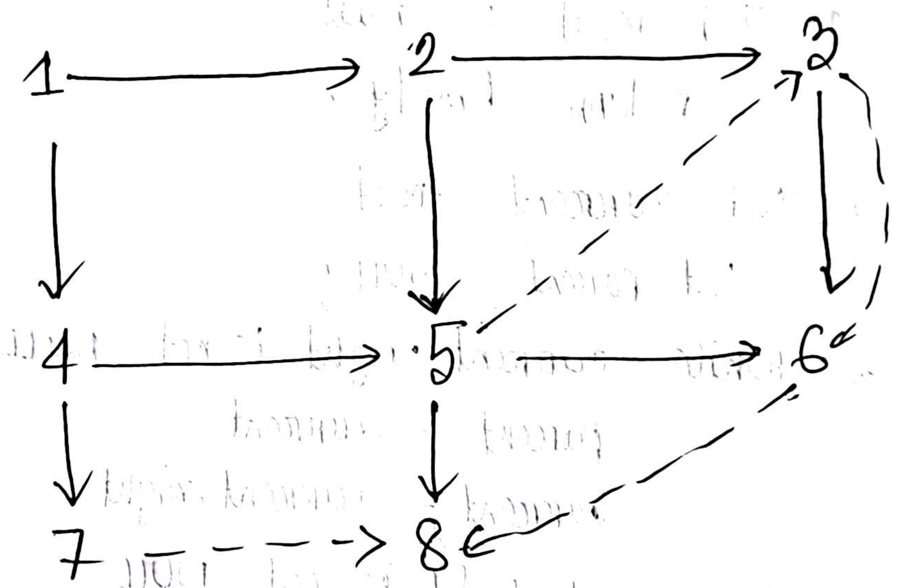
Now, Here is the Pseudocode of the Algorithm:

Algorithm SecondHeights(root)

1. If root is null
   return Empty;
2. Set current = root
   Set parent = NULL;
3. While current.right is not NULL
   parent = current
   current = current.right
4. If current.left is not NULL
   temp = curr.left
   while temp.right is not null
   temp = temp
   return temp.data.
5. Else, return parent data ;

Answer to the Question NO: 3(a)

The graph :



Thus the directed graph containes vertices
$\{1, 2, 3, 4, 5, 6, 7, 8\}$

Ans' to the Q. NO: 3(b)

Topological sorting gives an order of visiting rooms such that all unlocking conditions are satisfied before entering the room.

one valid order is

$1 \to 4 \to 5 \to 3 \to 6 \to 8 \to 2 \to 7$

In this order ensure that

- Room 5 visited before Room 3
- Room 3 visited before Room 6
- Room 6 visited before Room 8.

$\therefore$ Hence All rooms can be unlocked.

Answer to the Question No: 3(c)

Starting DFS from Room-1, the search goes as deep as possible before backtraking.

The path:

1 → 2 → ~~8~~ 3 → 6 → 8

∴ Number of moves = 4.

∴ DFS Reaches Room-8 by going deep along one path without exploring all neighbors frist.

Answer to the Question No; 3 (d)

Starting BFS from Room-1 the neighboring rooms are explored level by level.

Path: 1 → 4 → 5 → 3 → 6 → 8

∴ Number of Moves = 5.

Although DFS Reaches the destination in fewer moves, BFS is more Suitable for this scenario because it ensures a valid and systematic travarsal while respecting unlocking conditions and finding the sortest valid path.

Answer to the Q. No: 4(a)

Build-Heap is the process of converting an unsorted array into valid heap. In the case of max heap every parent node must be greater than or equal to its children.

However if the given array already satisfied the max-heap property, there is no need to be Build heap operation again. This because the heap priority is already maintained.

Ans to the Q. No: 4(b)

Given the array

23	56	11	87	34	75

Step 1: Build max heap

87	~~7~~56	75	23	34	11

Swap 87 and 11 :

11	56	75	23	34	87

Swap 75 and 34 :

34	56	11	23	75	87

Step 2: Heap Sort :

Finally:

11	23	34	56	75	87

Answer to the Q.No: 4(c)

If a node's priority decreases, it may become smaller than its children, violating max-heap's property. To fix this max-Heapify is applied to that node, restoring the heap structure effeciently.